

Lab 7 Report - Prolog Number Converter

May, 18th

Nolan Capomaggio & Mathis Morra-Fischer

Problem description

The goal of this exercise was to solve Variant 4 using Prolog.

The task was to convert English words of a written number into numerical digits. The number must be less than or equal to 1000.

For example:

```
?- to_num("ninety nine").  
99  
  
?- to_num("one hundred and one").  
101
```

The program supports numbers from 0 to 1000, including units, teens, tens, hundreds, and the special case `one thousand`.

Logic flow of the solution

The solution first receives an input string such as:

```
"one hundred and twenty-five"
```

The input is normalized before being parsed. The program:

1. Converts the input to lowercase.
2. Splits the string using spaces and hyphens.
3. Removes the optional word `"and"`.
4. Parses the cleaned list of words.
5. Computes the final numerical value.
6. Checks that the result is less than or equal to 1000.

For example:

```
"one hundred and twenty-five"
```

is converted into the list:

```
["one", "hundred", "twenty", "five"]
```

Then the program recognizes the structure:

```
one hundred + twenty five
```

and calculates:

```
1 * 100 + 25 = 125
```

Main components of the code

Word facts

The code defines several groups of facts:

```
unit("one", 1).  
teen("ten", 10).  
tens("twenty", 20).
```

These facts associate English words with their numerical values.

The words are separated into categories because not all combinations are valid. For example:

```
twenty five
```

is valid, but:

```
twenty ten
```

is not valid.

remove_and/2

This predicate removes the word "and" from the list of words.

For example:

```
["one", "hundred", "and", "five"]
```

becomes:

```
["one", "hundred", "five"]
```

This allows the program to accept both:

```
one hundred and five
```

and:

```
one hundred five
```

below_hundred/2

This predicate parses numbers from 1 to 99.

It accepts:

- units, such as one, two, nine
- teens, such as ten, eleven, nineteen
- tens, such as twenty, thirty, ninety
- tens followed by a unit, such as twenty five

Example:

```
below_hundred(["twenty", "five"], N).
```

returns:

```
N = 25.
```

This rule prevents invalid expressions such as:

```
six five  
ten ten  
twenty ten
```

parse_number/2

This predicate parses the full number.

It handles:

```
parse_number(["zero"], 0).
```

for zero,

```
parse_number(["one", "thousand"], 1000).
```

for one thousand,

and hundreds such as:

```
parse_number(["three", "hundred"], 300).  
parse_number(["three", "hundred", "twenty", "five"], 325).
```

For hundreds, the program uses:

```
N is U * 100 + R
```

where `U` is the value before "hundred" and `R` is the remaining part below 100.

to_num/2

This is the main logical predicate.

Example:

```
?- to_num("ninety nine", N).  
N = 99.
```

It returns the result in the variable `N`.

to_num/1

This predicate matches the example format from the exercise.

Example:

```
?- to_num("ninety nine").  
99
```

It calls `to_num/2` internally and prints the result.

Challenges

The main challenge was to avoid accepting invalid word combinations.

A simple approach would be to add the value of each recognized word. However, this would accept incorrect inputs such as:

```
six five  
ten ten
```

```
twenty ten
```

For example, a simple accumulator could calculate:

```
six five = 6 + 5 = 11
```

but this is not a valid English written number.

To solve this problem, the words were separated into categories: units, teens, and tens. Then, the code only allows valid structures, such as:

```
unit  
teen  
tens  
tens + unit  
unit + hundred  
unit + hundred + number below hundred  
one thousand
```

Another challenge was handling different writing styles. For example, the program should accept:

```
twenty five  
twenty-five  
one hundred and five  
one hundred five
```

This was solved by splitting the input using both spaces and hyphens, and by removing the optional word "and" .

Conclusion

The final solution converts English written numbers into numerical digits for values from 0 to 1000.

The program is stricter than a simple accumulator because it checks whether the sequence of words follows a valid number structure. This allows it to correctly reject invalid inputs such as "six five" or "twenty ten" .